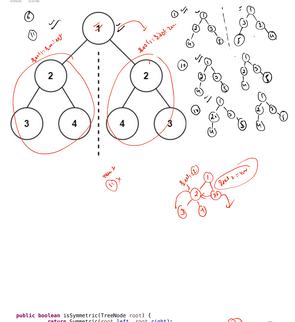
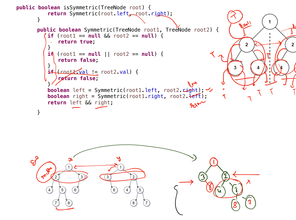
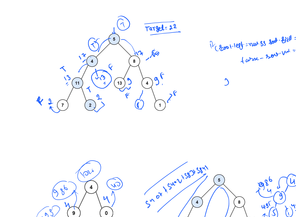
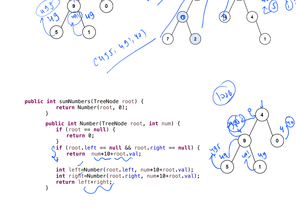


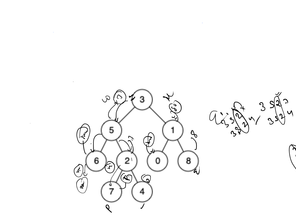
```
public boolean isSymmetric(TreeNode root) {
        return Symmetric(root.left, root.right);
    }

    public boolean Symmetric(TreeNode root1, TreeNode root2) {
        if (root1 == null && root2 == null) {
            return true;
        }

        if (root1 == null || root2 == null) {
            return false;
        }

        if(root1.val != root2.val) {
            return false;
        }

        boolean left = Symmetric(root1.left, root2.right);
        boolean right = Symmetric(root1.right, root2.left);
        return left && right;
    }
```



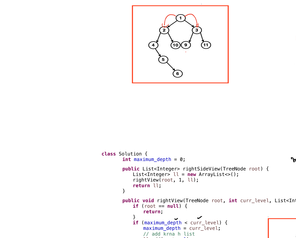


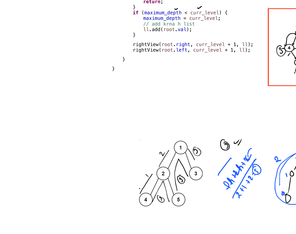


```
public int sumNumbers(TreeNode root) {
        return Number(root, 0);
    }

    public int Number(TreeNode root, int num) {
        if (root == null) {
            return 0;
        }

        if (root.left == null && root.right == null) {
            return num*10+root.val;
        }

        int left=Number(root.left, num*10+root.val);
        int right=Number(root.right, num*10+root.val);
        return left+right;
    }
```



```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) {
            return null;
        }
        if (root == p || root == q) {
            return root;
        }
        TreeNode left = lowestCommonAncestor(root.left, p, q);
        TreeNode right = lowestCommonAncestor(root.right, p, q);
        if (left != null && right != null) {
            return root;
        } else if (left == null) {
            return right;
        } else {
            return left;
        }
    }
```

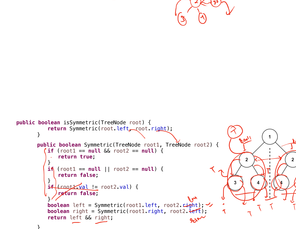




```
class Solution {
    int maximum_depth = 0;

    public List<Integer> rightSideView(TreeNode root) {
        List<Integer> ll = new ArrayList<>();
        rightView(root, 1, ll);
        return ll;
    }
    public void rightView(TreeNode root, int curr_level, List<Integer> ll) {
        if (root == null) {
            return;
        }
        if (maximum_depth < curr_level) {
            maximum_depth = curr_level;
            // add krna h list
            ll.add(root.val);
        }

        rightView(root.right, curr_level + 1, ll);
        rightView(root.left, curr_level + 1, ll);

    }
}
```

```
public int diameterOfBinaryTree(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int ld = diameterOfBinaryTree(root.left);
        int rd = diameterOfBinaryTree(root.right);
        int sd = bt(root.left) + bt(root.right) + 2;
        return Math.max(ld, Math.max(rd, sd));
    }

    private int bt(TreeNode node) {
        if (node == null) {
            return -1;
        }

        int lh = bt(node.left);
        int rh = bt(node.right);
        return Math.max(lh, rh) + 1;
    }
```

```
public int diameterOfBinaryTree(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int ld = diameterOfBinaryTree(root.left);
        int rd = diameterOfBinaryTree(root.right);
        int sd = bt(root.left) + bt(root.right) + 2;
        return Math.max(ld, Math.max(rd, sd));
    }

    private int bt(TreeNode node) {
        if (node == null) {
            return -1;
        }

        int lh = bt(node.left);
        int rh = bt(node.right);
        return Math.max(lh, rh) + 1;
    }
```



```
public DiaPair diameter(TreeNode root) {
        if (root == null) {
            return new DiaPair();
        }

        DiaPair ldp = diameter(root.left);
        DiaPair rdp = diameter(root.right);
        int sd = ldp.ht + rdp.ht + 2;
        DiaPair sdp = new DiaPair();
        sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
        sdp.d = Math.max(sd, Math.max(ldp.d, rdp.d));
        return sdp;
    }
}

class DiaPair {
    int d = 0;
    int ht = -1;
}
```